UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lecture 3: Design Choices

Important components of neural networks and how to make choices about which to choose.

Instructor: Thomas Lacombe

Learning outcomes

- **Distinguish** between hyperparameters and parameters in neural networks.
- **Identify** commonly used activation functions and **explain** their advantages and limitations.
- **Explain and compare** different weight initialisation strategies and their impact on training.
- **Explain** how regularisation techniques help prevent overfitting in neural networks.
- **Analyse** how architectural and training design choices influence the bias-variance trade-off.

Disclaimer/Acknowledgment:

Some of the following slides are reusing and adapting some of the content created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.

All CS188 materials are available at http://ai.berkeley.edu.

Some content is also reused from the CS230 Stanford course slides created by Andrew Ng, available at https://cs230.stanford.edu/syllabus/.

Extra resources

If you want to go further:

- *Hands-On Machine Learning with Scikit-Learn and PyTorch*, Géron A.
  - Chapters 9, 10 and 11.

- *Deep Learning*, Goodfellow I. et al.
  - Part II Deep Networks, chapters 7, 8 and 11.

Empirical and iterative process

Lots of choices to make!

- Evaluation:

Loss function

Evaluation strategy

Evaluation metrics

- NN hyperparameters:

Number of layers

Number of neurons per layer

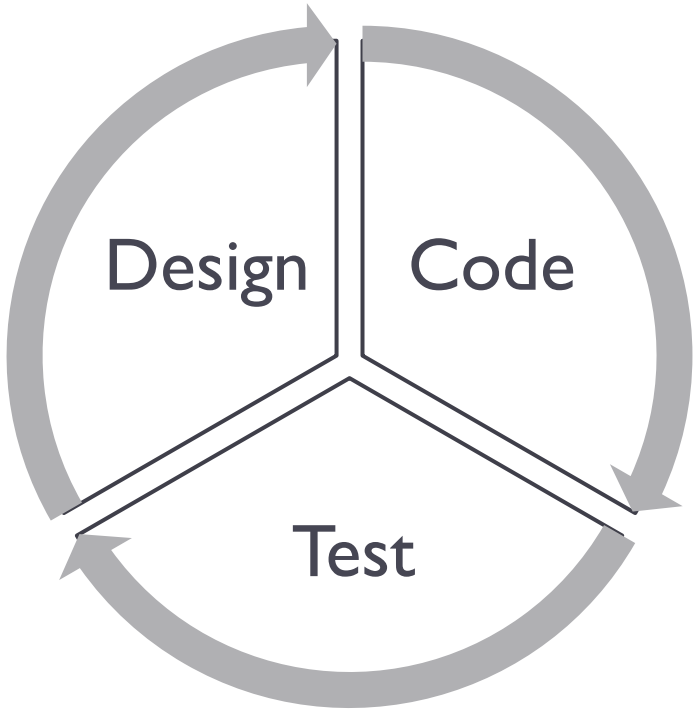
Activation functions

Learning rate

Weight initialisation

Mini-batch size

…

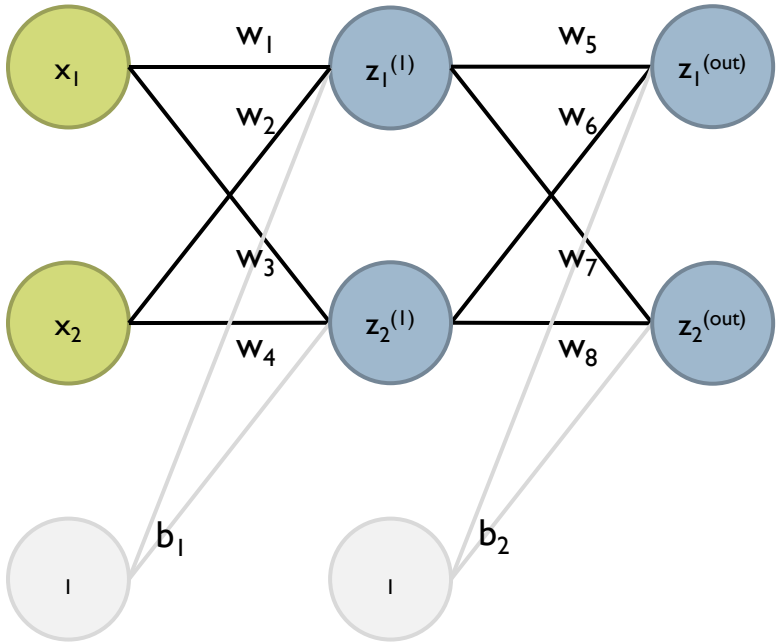


→ Eventually dealt with by AutoML with neural architecture search (NAS)?

https://dl.acm.org/doi/pdf/10.1145/3447582

Parameters vs Hyperparameters

- What are the parameters of a NN?



→ The weights (w_1, w_2, …) and the biases (b_1, b_2, …).

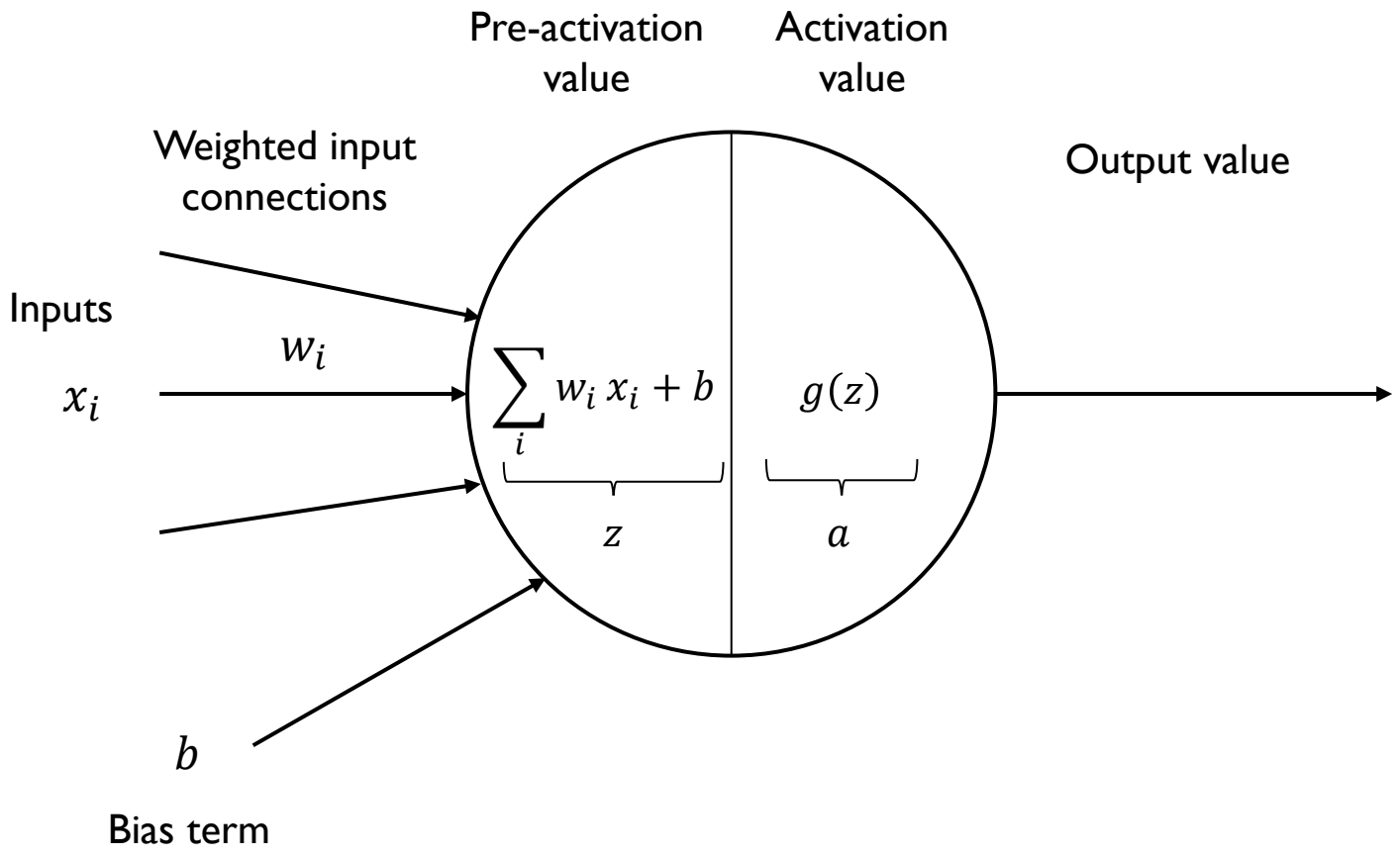
Parameters vs Hyperparameters

- What are the hyperparameters of a NN?

  - Learning rate (α)
  - Number of hidden layers
  - Number of neurons per layer
  - Number of iterations
  - Choice of the activation function
  - Choice of the optimisation method
  - Choice of regularisation
  - …

- They control the training process but are external to the model.
- They need to be tuned manually or automatically (e.g., meta-learning).

Artificial neuron



$$y = g\left(\sum_i w_i x_i + b\right)$$

Why non-linear activation functions?

- If no activation function, output of a neuron is:

$$\Sigma_i w_{i,j} x_i + b_j$$

  → Linear classifier (whatever number of neurons and layers)

- Activation functions introduce the non-linearity needed to model non-linear patterns.

Exploding and vanishing gradients

- Gradients are calculated in the backpropagation process to update the weights in the desired direction.

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} \cdot g'(z^{(L)}) \cdot g'(z^{(L-1)}) \cdots g'(z^{(1)}) \cdot x,$$

- $z^{(l)}$is the pre-activation at layer l,
- $g'(z^{(l)})$is the derivative of the activation function at layer l,
- x is the input connected to weight w.

Exploding and vanishing gradients

- Gradients are calculated in the backpropagation process to update the weights in the desired direction.

- **Vanishing gadients:**
  - Repeated multiplication of small derivatives
  - Gradient shrinks exponentially as L increases
  - Can slow down or stop the learning process (very small weights' update)

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} \cdot g'(z^{(L)}) \cdot g'(z^{(L-1)}) \cdots g'(z^{(1)}) \cdot x,$$

$$\| \frac{\partial \mathcal{L}}{\partial w} \| \to 0$$

Exploding and vanishing gradients

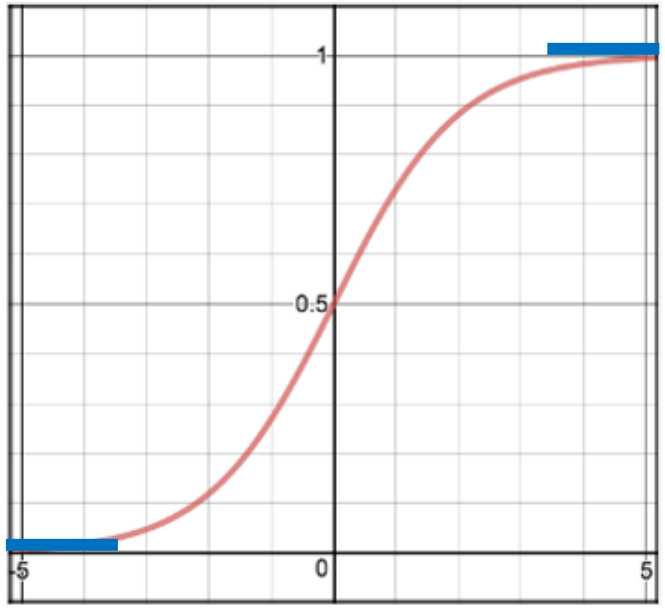
- Gradients are calculated in the backpropagation process to update the weights in the desired direction.

- **Exploding gradients:**
  - Repeated multiplication of large derivatives
  - Gradient grows exponentially
  - Learning can become unstable (large weights update) and diverge

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z^{(L)}} \cdot g'(z^{(L)}) \cdot g'(z^{(L-1)}) \cdots g'(z^{(1)}) \cdot x,$$

$$\| \frac{\partial \mathcal{L}}{\partial w} \| \to \infty$$

Quiz



Using this activation function for each hidden layer of a deep NN, what problem(s) might occur if the input values stay in the blue zones?

A. The gradients might explode.
B. The gradients might vanish.
C. The network might take a long time to train.
D. The computer used for training might crash.

What characteristics do we want for the activation function?

- **Vanishing gradient problem**: No gradient shifting towards 0.
- **Zero-centred**: Symmetrical to 0 so gradient does not go in a particular direction.
- **Computational inexpensive**: Activation function computed a lot of times (especially in large DNN!).
- **Differentiable**: To be able to calculate the gradient for the backpropagation in the gradient descent process.

Existing activation functions

Summary of common activation functions:

https://ml-cheatsheet.readthedocs.io/en/latest/activation_functions.html

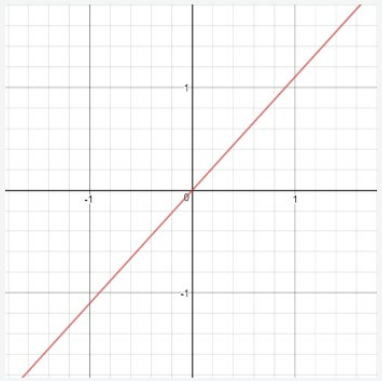
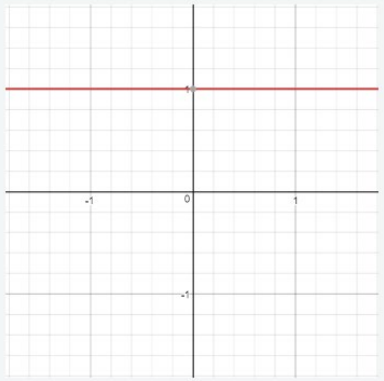
More advanced activation functions:

Hands-On Machine Learning with Scikit-Learn and PyTorch, pp.368-375

Linear function

Linear

A straight line function where activation is proportional to input (which is the weighted sum from neuron).

Function	Derivative
$R(z,m) = \{ z * m \}$	$R'(z,m) = \{ m \}$
	
`def linear(z,m):` `  return m*z`	`def linear_prime(z,m):` `  return m`

- Can only represent linear functions! Last layer will always be a linear function of first layer if used in hidden layers.

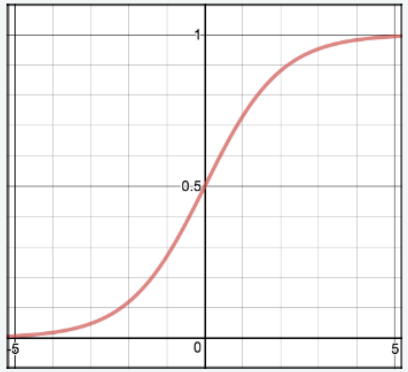
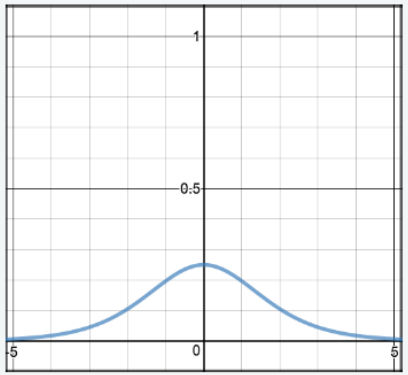
$$a_j = g(\Sigma_i w_{i,j} x_i + b_j)$$

- Can be used in the output layer for regression.

Sigmoid function

Sigmoid

Sigmoid takes a real value as input and outputs another value between 0 and 1. It's easy to work with and has all the nice properties of activation functions: it's non-linear, continuously differentiable, monotonic, and has a fixed output range.

Function	Derivative
$S(z) = \frac{1}{1+e^{-z}}$	$S'(z) = S(z) \cdot (1 - S(z))$
	
`def sigmoid(z):` `  return 1.0 / (1 + np.exp(-z))`	`def sigmoid_prime(z):` `  return sigmoid(z) * (1-sigmoid(z))`

Pros

- Non-linear.
- The output of the activation function is always going to be in range (0,1) compared to (-inf, inf) of linear function. So, we have our activations bound in a range. Prevents activation to blow up.
- Can be used to output a probability between 0 and 1.

Cons

- Towards either end of the sigmoid function, the Y values tend to respond very less to changes in X.
- It gives rise to a problem of "vanishing gradients".
- Its output isn't zero centred. Weights update less effective leading to "zig zag" learning.
- A bit computationally expensive.

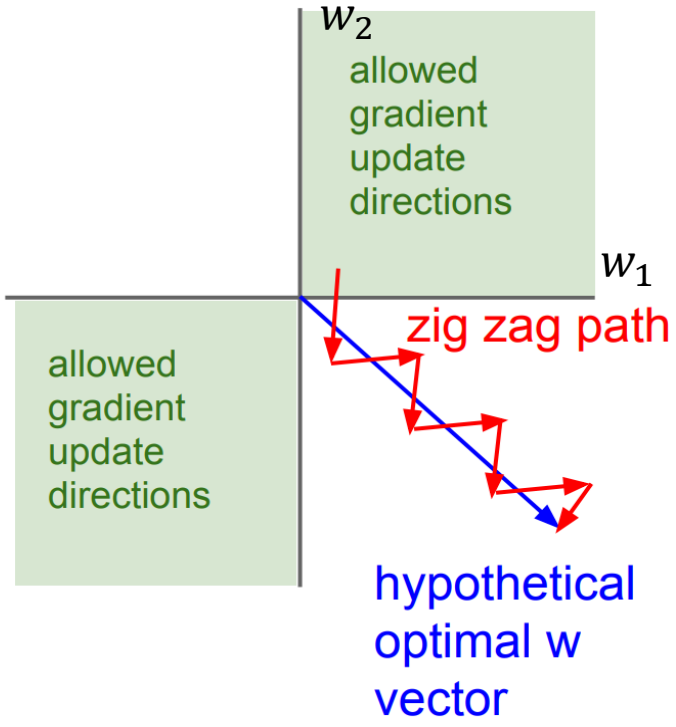
« Zig zag learning »

- What if all inputs $x_i > 0$ and $g'(\cdot) \geq 0$?

$$f = g(\Sigma_i w_i x_i + b_i)$$

$$\frac{df}{dw_i} = g'(\Sigma_i w_i x_i + b_i) * x_i$$

$$\frac{dL}{dw_i} = \frac{dL}{df}\frac{df}{dw_i} = \frac{dL}{df} * g'(\Sigma_i w_i x_i + b_i) * x_i$$



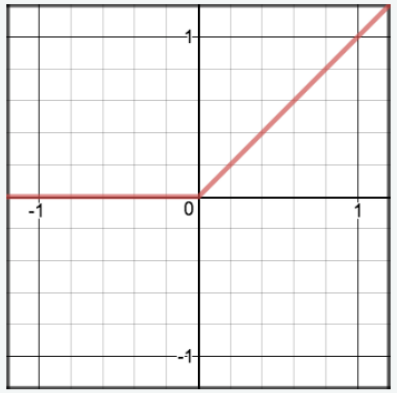
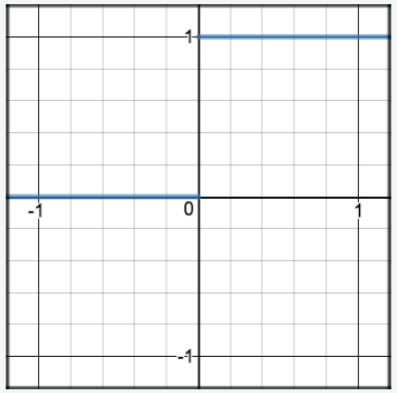
- $\frac{dL}{dw_i}$ solely determined by $\frac{dL}{df}$ (upstream gradient)
- All weights gradient have the same sign
- All weights updated in the same direction (either all increase or all decrease)
- That is also why normalised zero-mean input data is preferred in a NN

Image taken from the CS 231n lecture slides

ReLU function

ReLU

A recent invention which stands for Rectified Linear Units. The formula is deceptively simple: $max(0, z)$. Despite its name and appearance, it's not linear and provides the same benefits as Sigmoid (i.e. the ability to learn nonlinear functions), but with better performance.

Function	Derivative
$R(z) = \begin{Bmatrix} z & z > 0 \\ 0 & z <= 0 \end{Bmatrix}$	$R'(z) = \begin{Bmatrix} 1 & z > 0 \\ 0 & z < 0 \end{Bmatrix}$
	
`def relu(z):` `  return max(0, z)`	`def relu_prime(z):` `  return 1 if z > 0 else 0`

Deep Sparse Rectifier Neural Networks Glorot et al., (2011)

Pros

- Avoids the vanishing gradient problem.
- Less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations.
- Faster learning as the slope does not go to 0.

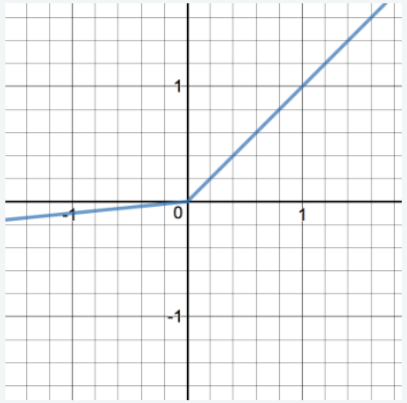
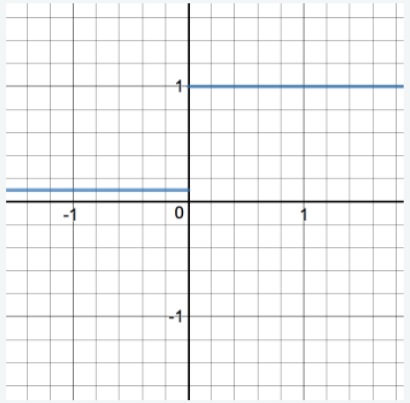
Cons

- Should only be used within hidden layers of a NN.
- "Zig zag" learning.
- *Dying ReLU problem*: output and gradient = 0 if input < 0. If a weight update makes the input always negative, neuron will "die", i.e., always return 0. Weight will stop being adjusted in the descent process and if too many neuron die, the network stops to learn.
- Range is [0,∞). This means it can blow up the activation.

Leaky ReLu function

LeakyReLU

LeakyRelu is a variant of ReLU. Instead of being 0 when $z < 0$, a leaky ReLU allows a small, non-zero, constant gradient α (Normally, $\alpha = 0.01$). However, the consistency of the benefit across tasks is presently unclear. [1]

Function	Derivative
$R(z) = \begin{cases} z & z > 0 \\ \alpha z & z <= 0 \end{cases}$	$R'(z) = \begin{cases} 1 & z > 0 \\ \alpha & z < 0 \end{cases}$
	
`def leakyrelu(z, alpha):` `  return max(alpha * z, z)`	`def leakyrelu_prime(z, alpha):` `  return 1 if z > 0 else alpha`

Leaky ReLUs are one attempt to fix the “dying ReLU” problem by having a small negative slope (of 0.01, or so).

Output layer activation functions

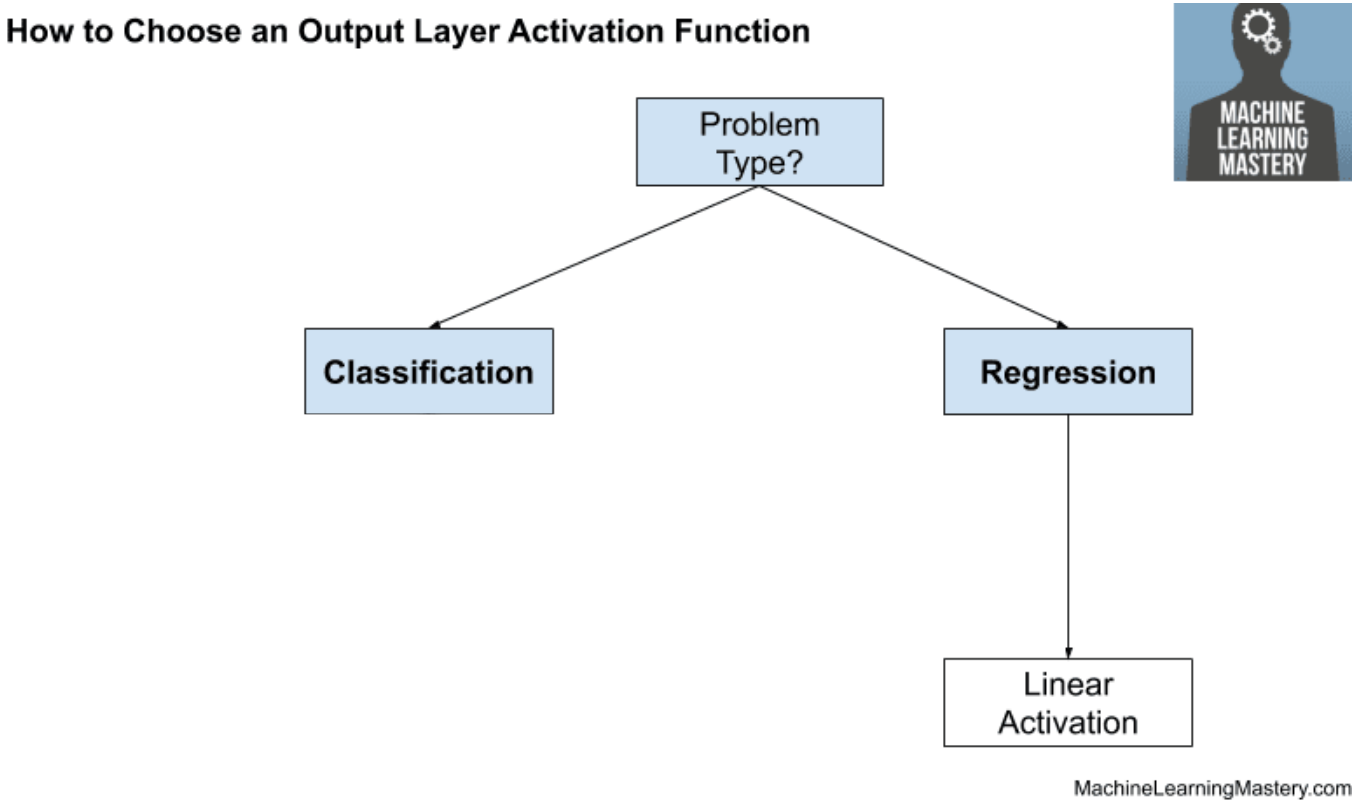


Image source: https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/

Output layer activation functions

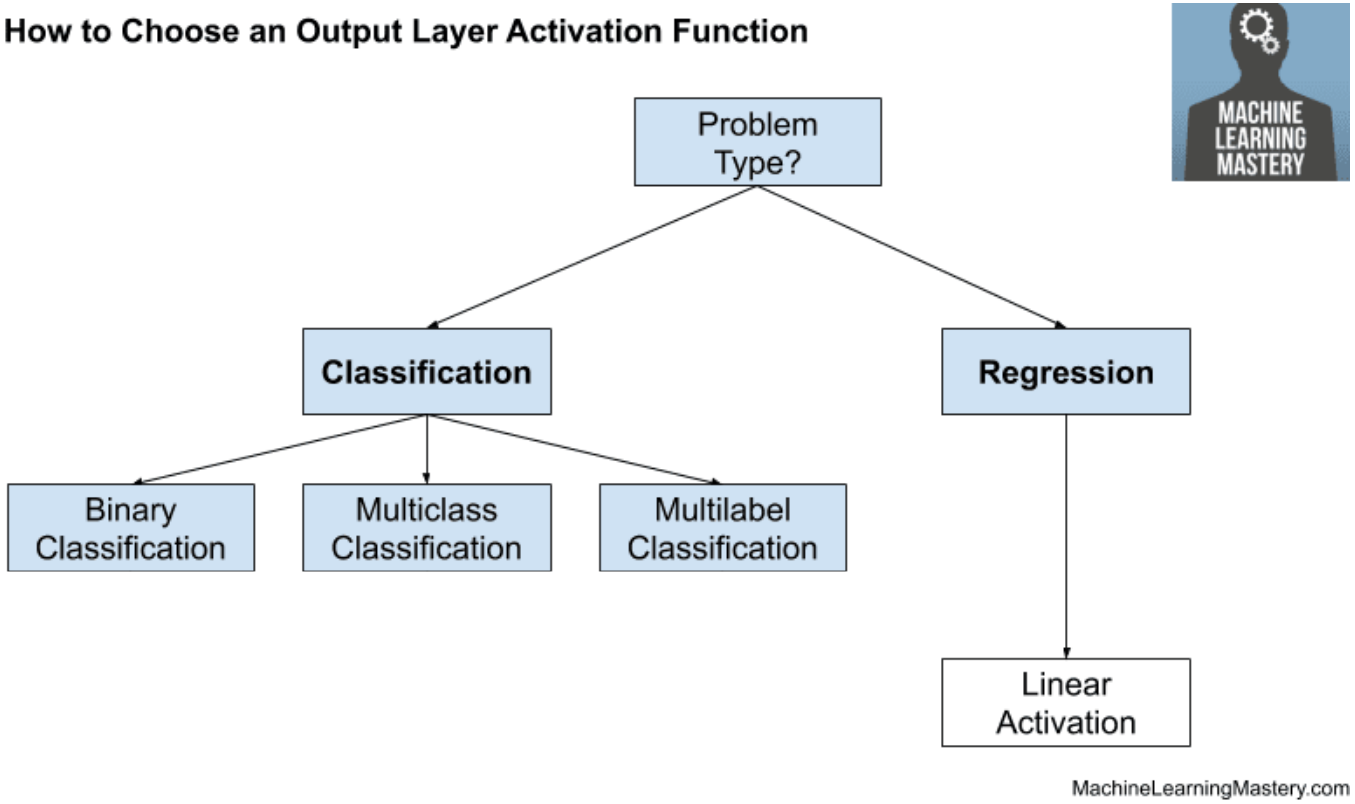


Image source: https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/

Logits

- Logit → raw output value produced by the last linear layer before any activation function

- For an input representation h from the previous layer:

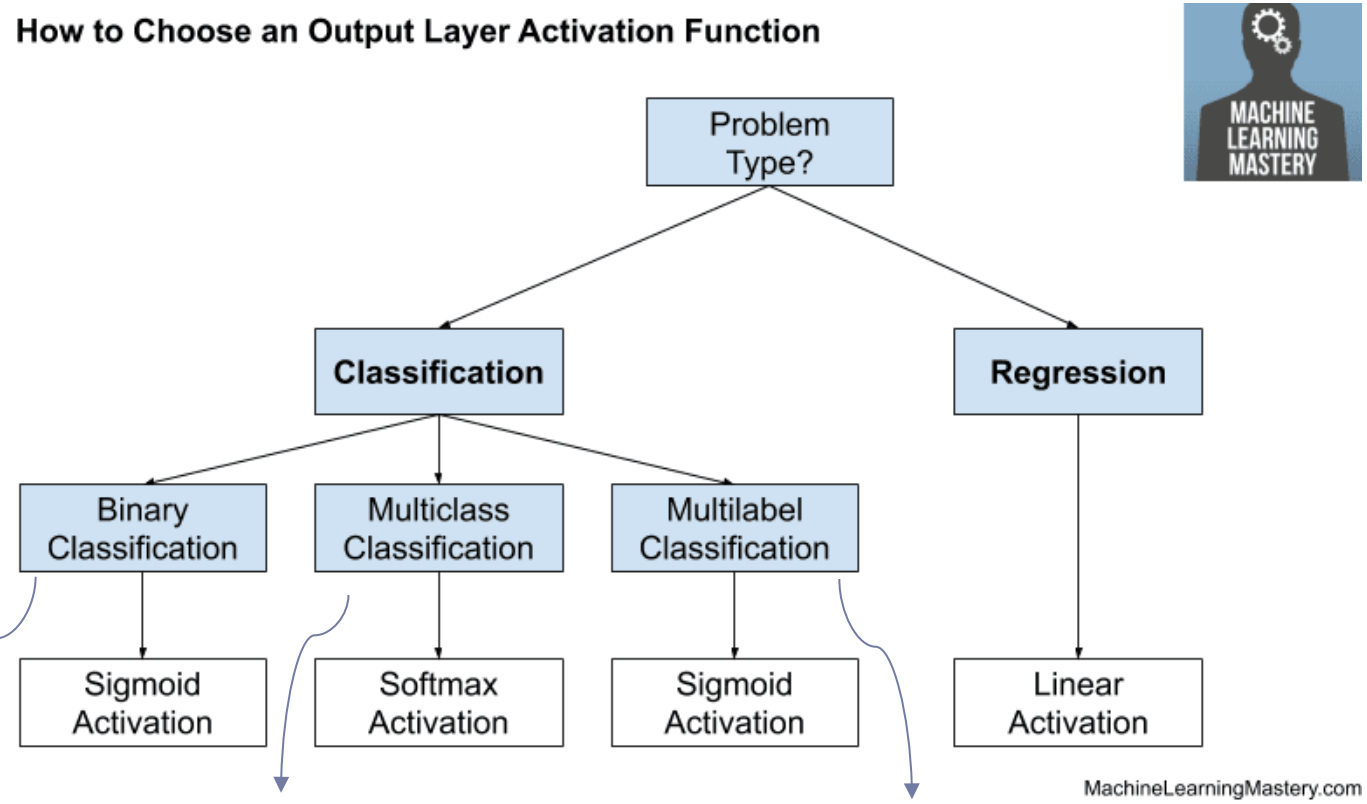
$$z = Wh + b$$

  - z: logits (one value per class)
  - W, b: weights and bias of the output layer

- Logits can take any real value $(-\infty, +\infty)$
- Larger logit → higher predicted likelihood for that class

- Become probabilities after applying an output activation function

Output layer activation functions



Is it a dog?

Is it a dog, a cat or a lion?

Is there a dog, a cat or a lion? (several possible)

Image source: https://machinelearningmastery.com/choose-an-activation-function-for-deep-learning/

Softmax function

- Multi-class classification problems

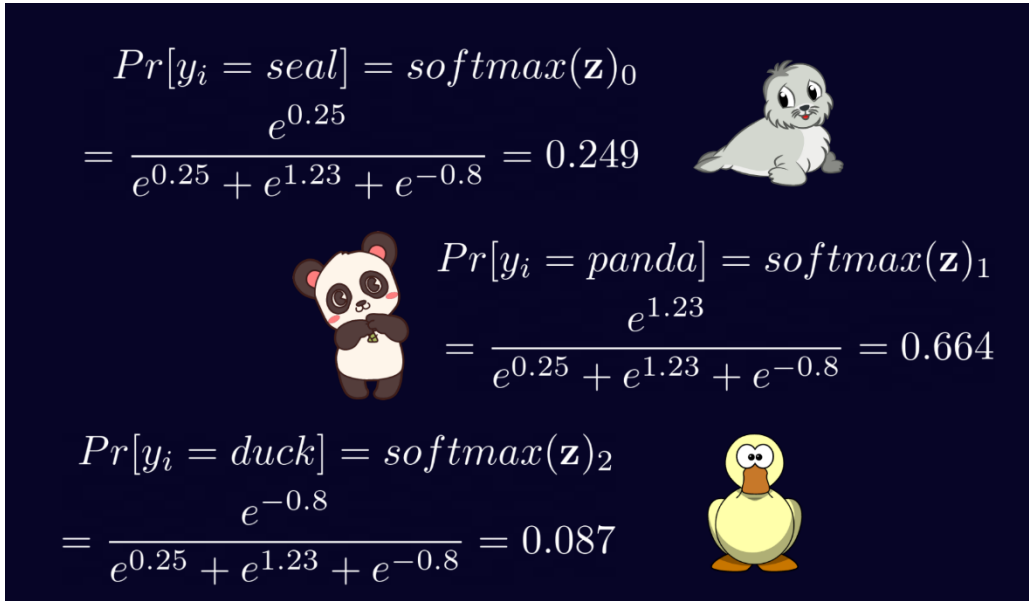
Class	Value
0	
1	
2	

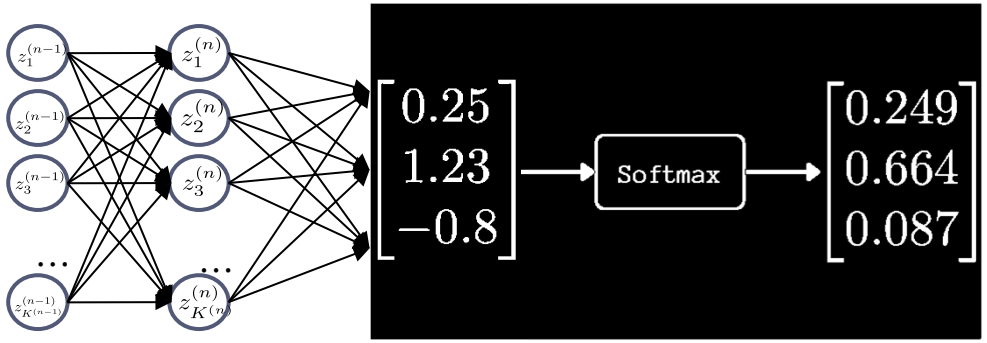
Images source: https://www.pinecone.io/learn/softmax-activation/

Softmax function

- Softmax formula:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$



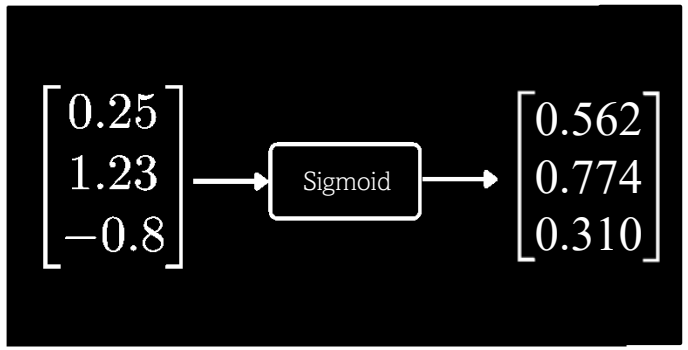


Credit images: https://www.pinecone.io/learn/softmax-activation/

Multi-class vs multi-label

- Sigmoid function

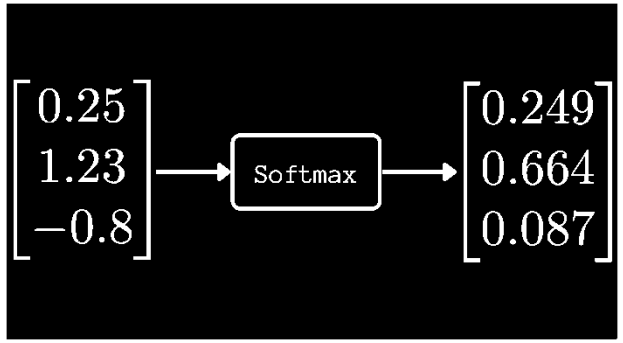
$$S(z) = \frac{1}{1 + e^{-z}}$$



- Use sigmoid if classes are not mutually exclusive (multi-label classification).

- Softmax function

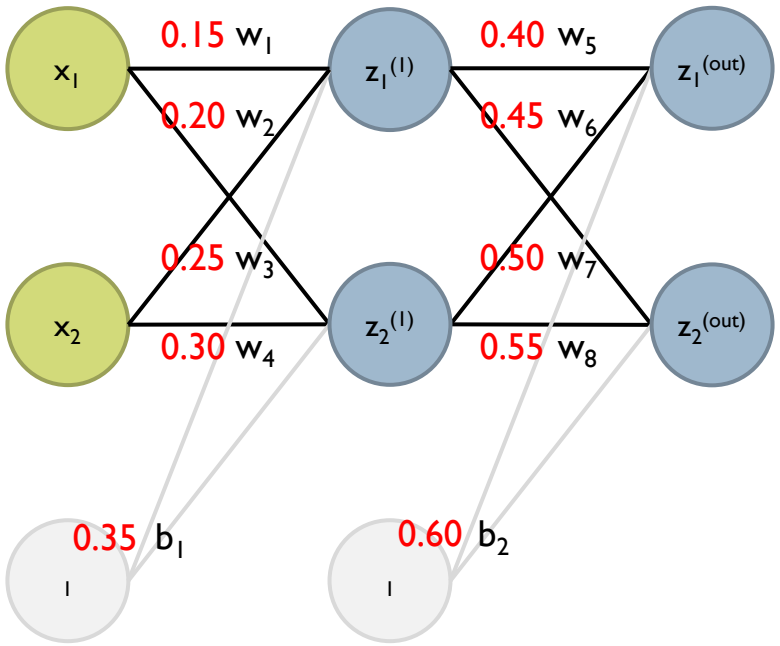
$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$



- Use softmax if classes are mutually exclusive (multi-class classification).

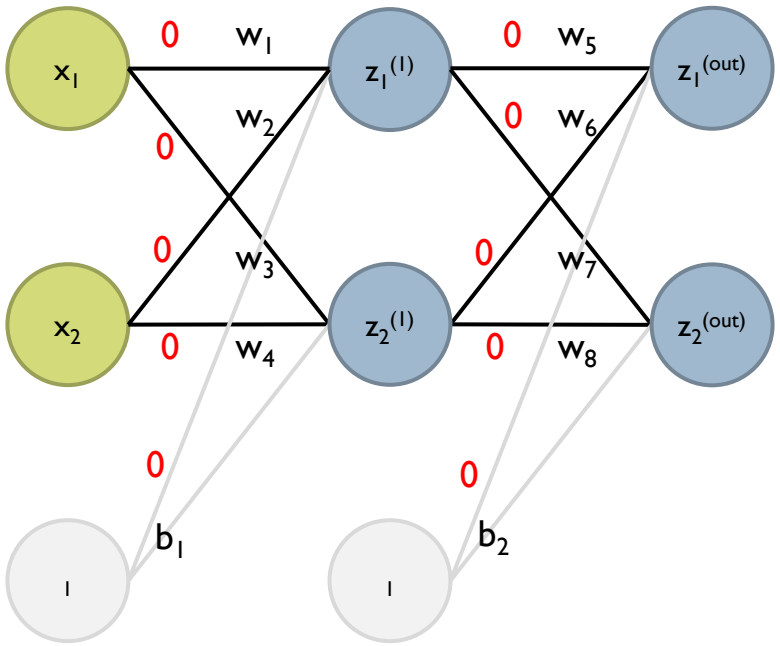
Network initialisation

- Initialisation of the weights



Network initialisation

- What if you initialise the weights of a NN to 0?



- Same problem with any constant value!

Random initialisation

- Always initialise weight randomly (e.g. standard normal distribution) to break the « symmetry ».

- What distribution to draw the weights from?

- Glorot and Bengio[1] showed that:
  - When using sigmoid activation + weights $w \sim \mathcal{N}(0,1)$
  - Variance of layer outputs is greater than variance of layer inputs
  - Problem: as variance of outputs increase through the model, sigmoid saturates and leads to small gradients (→ vanishing gradients)

[1] Xavier Glorot and Yoshua Bengio, "Understanding the Difficulty of Training Deep Feedforward Neural Networks." *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics* (2010): 249-256

Xavier (Glorot) initialisation

- Goal
  - Keep signal magnitudes approximately constant accross layers by maintaining same variance for input and outputs of each layer
- Only possible if layer has same number of inputs and outputs!
- Solution:
  - Compromise working well in practice: initialise the weigths from

$$\mathcal{N}\left(0, \frac{1}{fan_{avg}}\right) \text{ with } fan_{avg} = \frac{fan_{in} + fan_{out}}{2}$$

$$\text{Or } \mathcal{U}\left[-\sqrt{\frac{3}{fan_{avg}}}, \sqrt{\frac{3}{fan_{avg}}}\right]$$

He (Kaiming) Initialisation

- Goal
  - ReLU activation zero out approximately half of the inputs, reducing the variance of a activations
  - Compensate for this by increasing initial weight variance

- Only possible if layer has same number of inputs and outputs!

- Solution:
  - In practice, initialise the weigths from

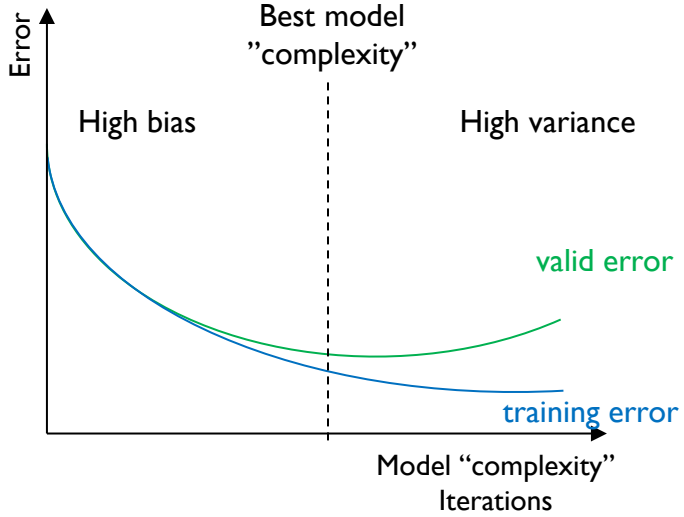
$$\mathcal{N}\left(0, \frac{2}{fan_{in}}\right)$$

$$\text{Or } \mathcal{U}\left[-\sqrt{\frac{6}{fan_{in}}}, \sqrt{\frac{6}{fan_{in}}}\right]$$

Kaiming He et al., "Delving deep into rectifiers: Surpassing human-level performance on imagenet classification", *Proceedings of the IEEE international conference on computer vision* (2015): *p. 1026-1034*

Regularisation techniques

- Very common problem with Deep Learning: overfitting



- Regularisation = discouraging learning a more complex model
- ↳ Reduces the variance, while trying not to increase the bias

Regularisation

"Regularization is any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error."

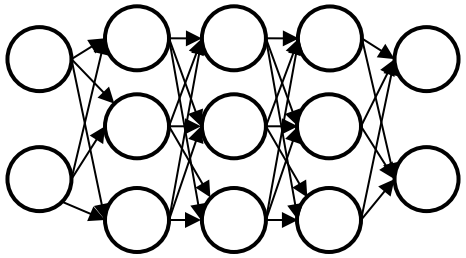
Deep learning, 5.2.2, p.117

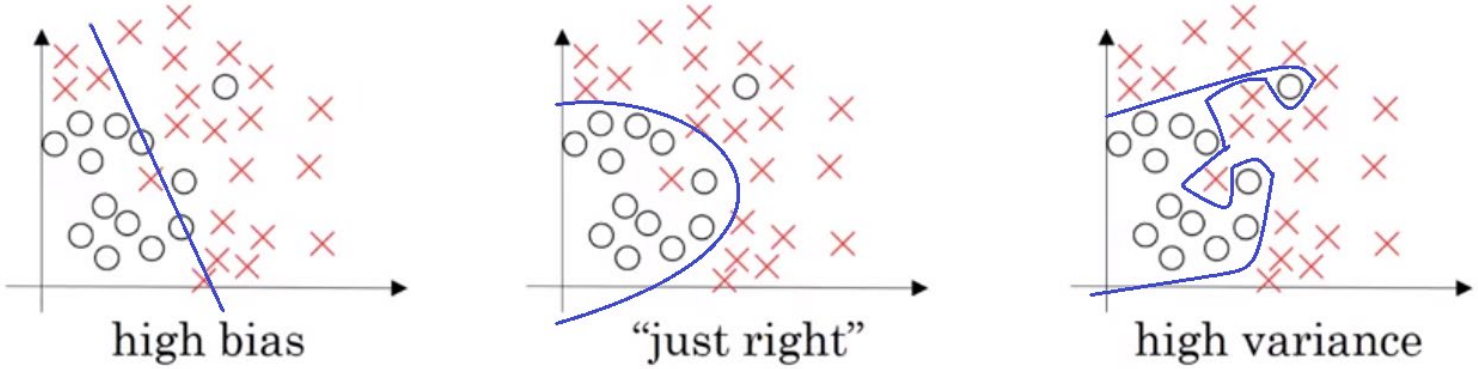
- Different techniques:
  - L1 & L2 regularisation
  - Dropout
  - Early stopping
- Data augmentation: can help indirectly

How does regularisation help to avoid overfitting?

First intuition:

- E.g., L1 & L2 regularisation penalises large weights values.
- Keeping weights close to zero for some neurons.

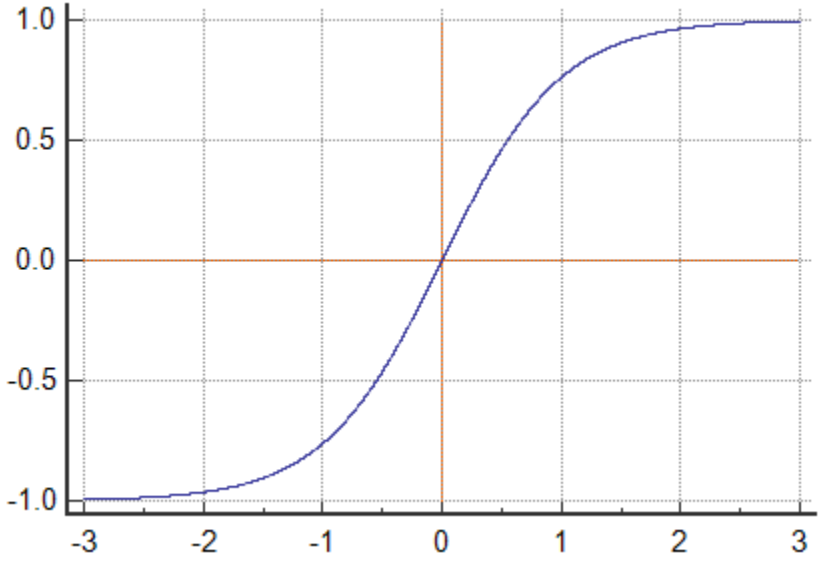




How does regularisation help to avoid overfitting?

Second intuition:

- Limiting the weights values will bring the output of a neuron in the linear zone of activation function (e.g. tanh).
- It will limit the NN power to model non-linearities.



L1 and L2 regularisation

- Why use it?
  - Large weights:
    - are characteristic of more complex models (higher learning time).
    - can be a sign of an over-specialized network (overfitting).
    - make the network unstable (sensitive to noise).
  - Penalises/constrains the weight values towards 0.
    - ↳ Encourages small weights.
    - ↳ Always prefer simpler models.

L1 and L2 regularization

How does it work?

1. Calculate the weights size
   - Sum of the absolute values of the weights → L1.
   - Sum of the squared values of the weights → L2.

L1 and L2 norms

$$\sum_{i=1}^{N} |w_i| \qquad ||\mathbf{w}||_1 = |w_1| + |w_2| + ... + |w_N|$$

$$\sum_{i=1}^{N} w_i^2 \qquad ||\mathbf{w}||_2 = \left(|w_1|^2 + |w_2|^2 + \ldots + |w_N|^2\right)^{\frac{1}{2}}$$

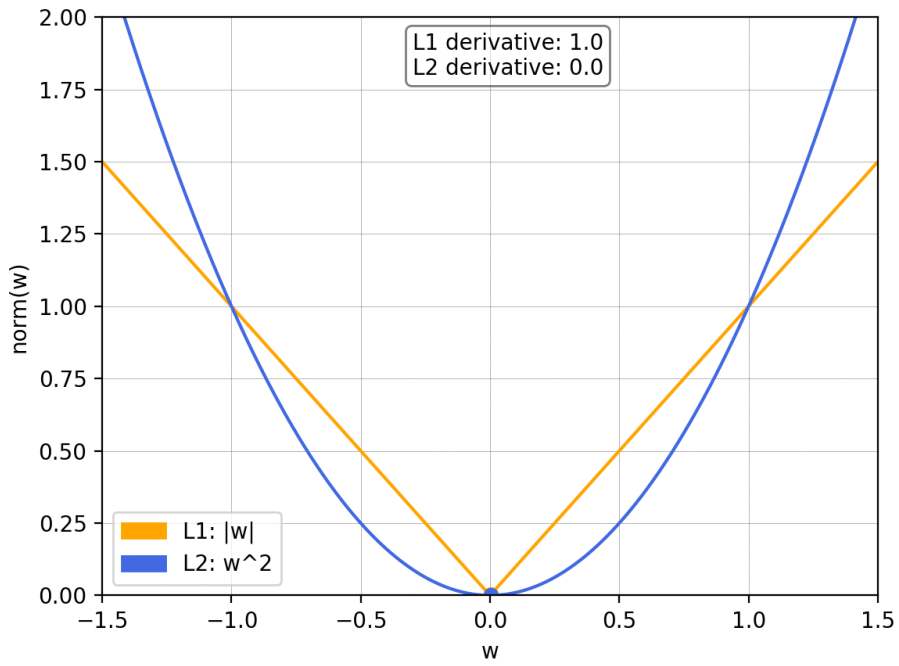
2. Apply regularisation to the weight update
   - L1 regularisation: $$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} |w_i|$$
   - L2 regularisation: $$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} w_i^2$$

Control the penalty

$$0 < \lambda < 1$$

L1 vs L2 regularisation

- Weight update: $w \leftarrow w - \frac{dL}{dw}$
- L1: $$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} |w_i|$$
- L2: $$Loss = Error(y, \hat{y}) + \lambda \sum_{i=1}^{N} w_i^2$$



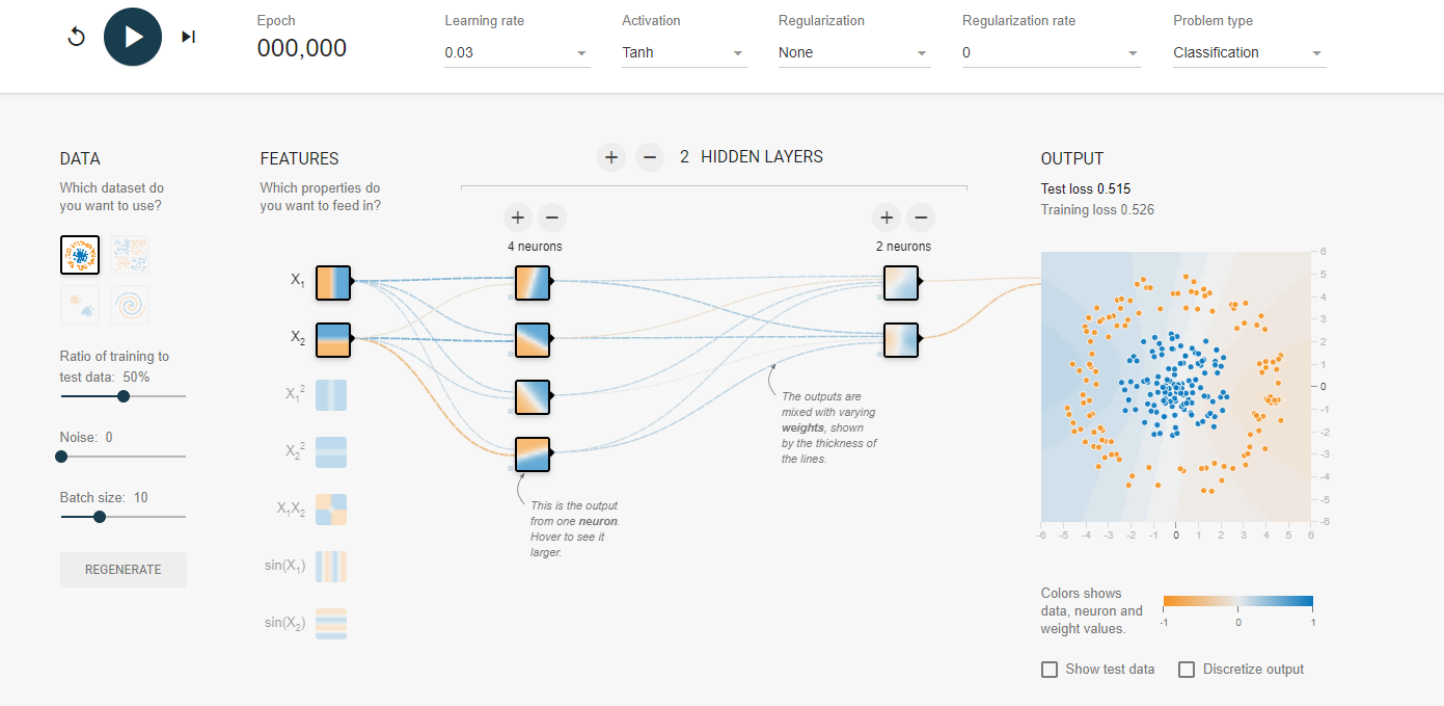
→ L2 penalises more large weights and less small weights → leads to less sparsity (more stable and effective for dense NNs).

→ L1 penalises all weights more evenly → leads to more sparsity (more weights = 0 → implicit feature selection).

Gif source:: https://towardsdatascience.com/visualizing-regularization-and-the-l1-and-l2-norms-d962aa769932

Tensorflow playground platform

- Interactive platform to test and visualise the effects of varying hyperparameters: https://playground.tensorflow.org/



Further reading about L1 and L2

- Intuition with step by step calculation:

https://towardsdatascience.com/intuitions-on-l1-and-l2-regularisation-235f2db4c261#15c2

- A more technical explanation:

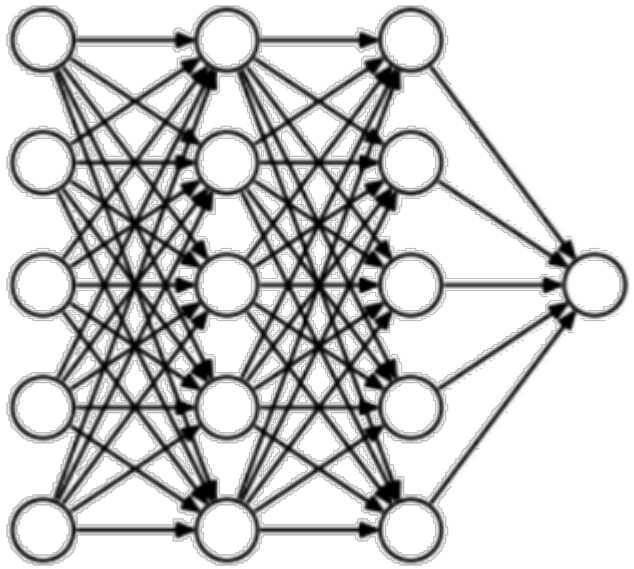
https://explained.ai/regularization/L1vsL2.html

- A more applied explanation:

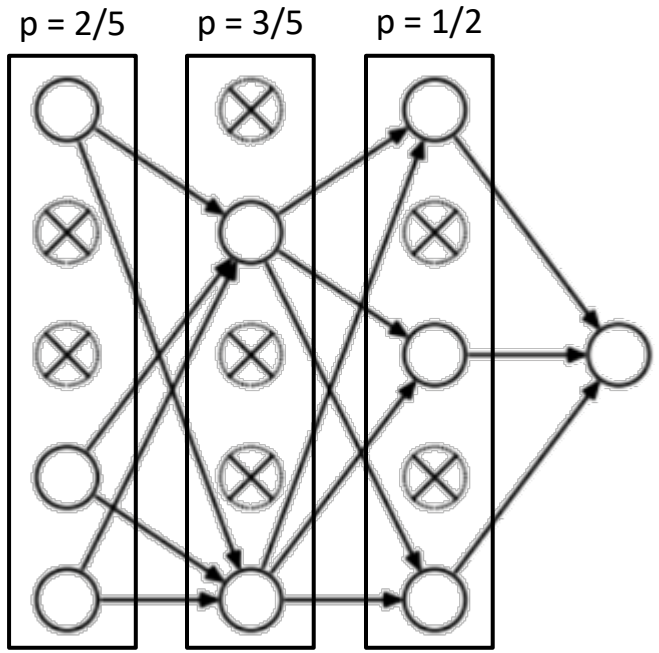
https://neptune.ai/blog/fighting-overfitting-with-l1-or-l2-regularization

Dropout

- How does it work?



Probability for hidden layers: p = 0.5
Probability for input layer: 0.5 < p < 1.0



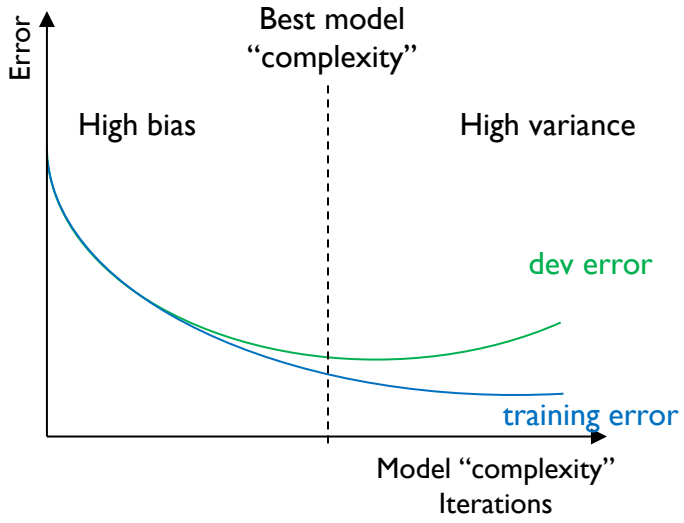
Dropout

- Why does it work?
  - The nodes cannot rely on any single previous node (feature).
    - Prevents too large weights.
    - Encourages spreading out the weights.
    - Forces the nodes to be more generally useful.
    - Introduces some noise in the activation values!

Can be combined with other forms of regularisation.

Early stopping

- Simple and popular regularisation technique.



- Learn enough, but not too much!

Avoid to end up in the high variance zone.

Early stopping

- When to stop?

1. Monitor the performance
   - Loss on the validation dataset.
   - Additional metrics (e.g., precision, recall, etc).
2. Trigger the early stopping
   - Simplest trigger: increase of the loss compared to the last iterations.
   - More elaborated ones: no change over several epochs, absolute change in a metric, average change in a metric over several epochs, reaching a specific level of performance, etc.
3. Choose the model to keep
   - Usually, keep the model from the epoch before the increase in loss.

Data augmentation

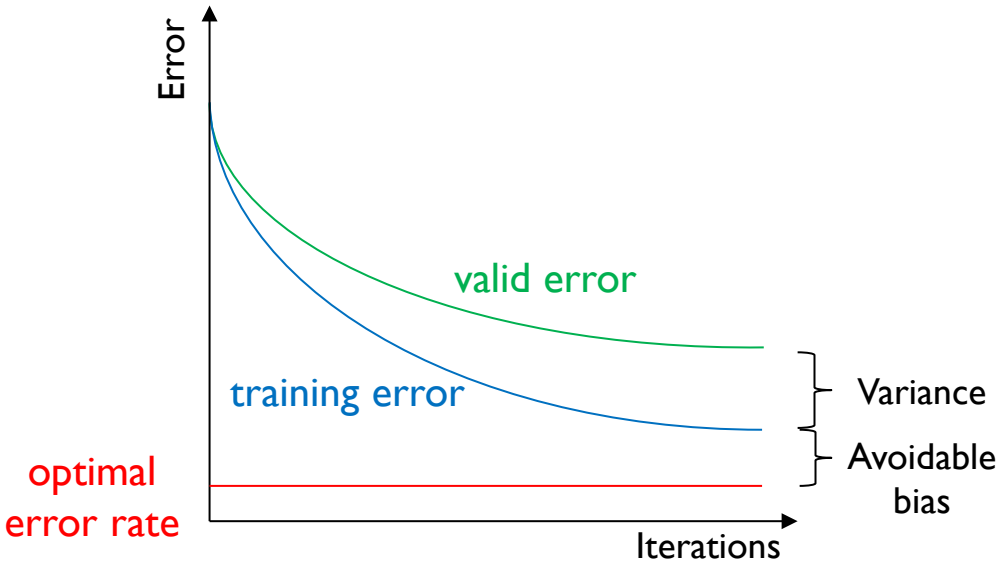
- Overfitting can happen if you do not have enough data to train all parameters.

- Pre-processing techniques → do not modify explicitly the learning algorithm.
  - Increase the training data set size.
  - Increase the diversity of the data.
  - Especially used with images.
  - Include operations such as rotating the image, flipping, scaling, adding noise, etc.

⚠ Can lead to underfitting if generated data not relevant to the task.

Optimal error rate / avoidable bias

- Optimal error rate
  - Error rate of an optimal classifier (e.g., human performance)
  - Can be hard to estimate
- Avoidable bias
  - Training error – optimal error rate
- Variance
  - Validation error – training error



Andrew Ng, "Machine Learning Yearning", Section 22.

Simplest formula to address variance/bias issues

- High avoidable bias
  - Intuition: model not complex enough to map inputs and outputs.
  - Simple fix: Increase model size (e.g., increase layers or neurons per layer).
  - Might increase variance and risk of overfitting (if no regularisation).
  - Will slow the learning.

- High variance
  - Intuition: training data not sufficient to generalise on validation data.
  - Simple fix: Add data to the training set.
  - More data might not be available.
  - Try data augmentation.

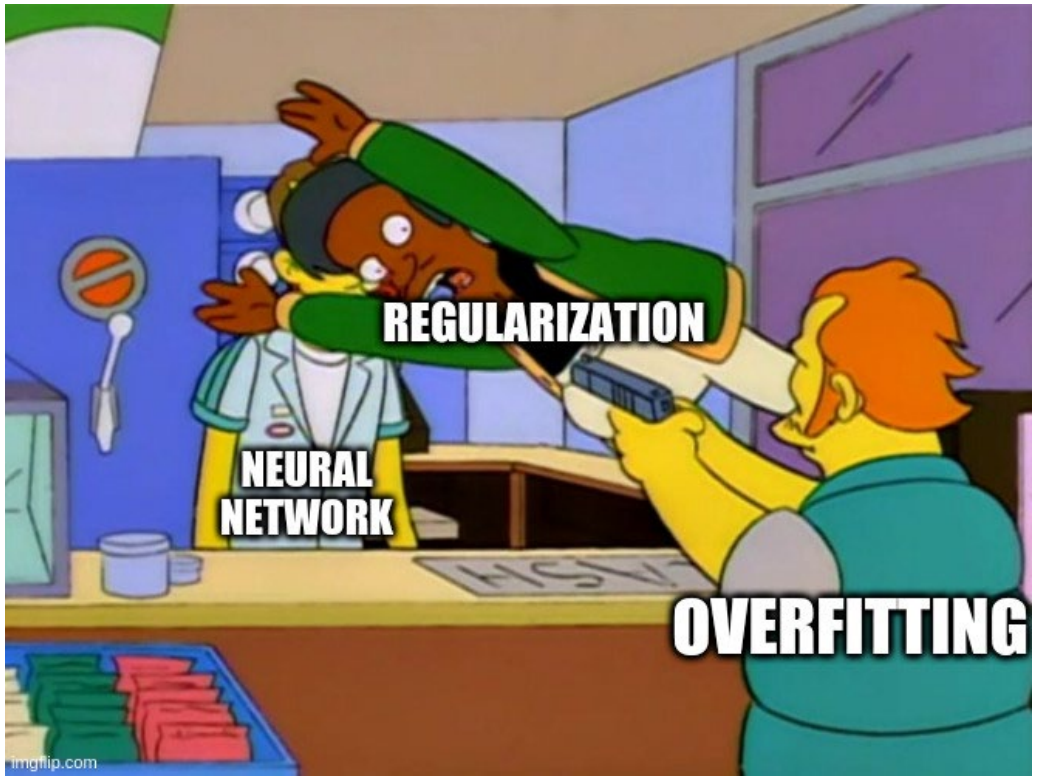
Andrew Ng, "Machine Learning Yearning", Section 23.

Bias vs variance tradeoff

- Some choices reduce bias but increase variance.
  - E.g., increasing size of the network.

- Some choices reduce variance but increase bias.
  - E.g., adding regularisation (early stopping might stop learning before reaching low bias, penalizing high weights might prevent the model to reach low bias, etc).
  - Effect of regularisation on bias can be reduced with a good hyperparameter tuning.
  - Data augmentation does not increase bias if relevant augmentation.

- More useful advice in Sections 25 to 27 of Andrew Ng's "Machine Learning Yearning" book, to reduce variance and bias.

Andrew Ng, "Machine Learning Yearning", Sections 24 to 27.

Conclusion memes